

P09 On Hold accounts and platform payouts

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P08.

1. What to build, in one sentence

Build On Hold accounts, where a bill is charged to a named account and collected later through a ledger, and platform payouts, where money the platforms send in batches is recorded and compared with what was expected, exactly as P07 specified them.

2. Module

The On Hold part of M16, and the payouts part of M17.

3. The spec is already written

`docs/API-CONTRACT.md` sections "M16 Settlement and Day Close" and "M17 Delivery and Platform Orders", and

`docs/DB-SCHEMA.md` sections 21, 22 and 23.

Build exactly what they say. If the spec and this prompt

disagree, the spec wins, and you tell me.

If the spec is silent on something you need, stop and ask.

Both features are "money that arrives later". That is why they share a prompt.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P09-on-hold-accounts-payouts.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P08 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P08's "after" count. If anything fails, stop and tell me.

5. Files to read first

1. The On Hold and payouts parts of the spec, as above.
2. `docs/GLOSSARY.md` sections 5, 6 and 7.
3. `docs/TEST-DATA.md` bills B09 and B10, and section 5, the next day.
4. `docs/RECONCILIATION-RULES.md` C3, C10 and C11.
5. `server/models/Bill.js` ,
`server/services/billService.js` ,
`server/services/billPermissionService.js` .
6. `server/services/paymentMethodService.js` from P08.

7. `server/utils/money.js` , especially `applyBasisPoints` .
 8. `server/utils/time.js` , with the test clock from P08.
-

6. Part A. On Hold accounts

Build, exactly as specified:

1. The `Account` and `AccountEntry` models, registered in `server/models/index.js` .
2. `server/services/accountService.js` , the one place that writes `accountentries` . It has `outstandingFor(req, accountId, { session })` , computed from the entries, never stored as a running number that could drift.
3. Creating an account with an opening balance writes the `OPENING` entry in the same transaction.
4. All the account endpoints, with their permissions.
5. `POST /bills/:billId/charge-to-account` , in one transaction: the bill becomes `ON_ACCOUNT` , the order becomes `BILLED` and frees its table exactly as the payment path does today, a `CHARGE` entry, and the `BILL_CHARGED_TO_ACCOUNT` audit line.
6. Voiding an `ON_ACCOUNT` bill writes a `CHARGE_REVERSED` entry in the same transaction as the void.
7. Collections: `IN_HAND` methods only, never more than the outstanding balance, frozen method fields, and a business date from `nowUtc()` .

8. Adjustments, owner only, with the

`ACCOUNT_BALANCE_ADJUSTED` audit line.

9. The statement: balance before `from` , each entry in order with a running balance, balance after `to` .

`ON_ACCOUNT` everywhere else.

Search the server for every use of the bill status constants and decide what `ON_ACCOUNT` means there:

Place	<code>ON_ACCOUNT</code> behaves like
Taking a payment	Refused: the bill is settled by the account
Applying a discount	Refused, like a paid bill
Voiding	Allowed, with the reversal above
Bill lists and filters	Its own status, shown as "On Hold"
Existing M6 sales figures	A sale, like any non-voided bill
Existing M6 payment figures	Not a payment. On Hold money is not received money.

List every place you changed, in your summary.

7. Part B. Platform payouts

Build, exactly as specified:

1. The `PlatformPayout` model, registered.

2. `server/services/payoutService.js` with `expectedFor(req, payout)`, which returns `{ expectedInPaise, includedPaymentCount, rateNotSet: [payments] }`, using each payment's frozen `commissionBps` and `applyBasisPoints`. Never a live rate.
3. The three endpoints. Overlapping live periods for one method are refused with 409 `PAYOUT_PERIOD_OVERLAP`. Voiding needs a reason and keeps the record.
4. The `PLATFORM_PAYOUT_RECORDED` audit line.
5. The list endpoint returns, for each payout, its expected amount and the difference, received minus expected.

Rounding. Expected payout per payment is the payment amount times `(10000 - commissionBps)` basis points, through `applyBasisPoints`, which rounds half away from zero. The batch expected is the sum of those per-payment figures. Write that sentence into the M17 spec section if P07 did not, so the R6 report later uses the same rounding.

8. The client

1. **Charge to account.** On an unpaid bill, a MANAGER or OWNER sees "Charge to account". It opens a searchable list of active accounts with their outstanding balance, and "Add account" for a new one.
2. **Accounts screen.** For OWNER and MANAGER, and CASHIER for collections: each account with its outstanding balance and oldest uncollected date, sorted by outstanding, highest first. Opening an

account shows its statement, with "Record collection" and, for the owner, "Adjust balance".

3. **Payouts screen.** For OWNER and MANAGER, under a "Money" heading: payouts per platform method, newest first, each with period, received, expected and the difference coloured with the `mirch` token when negative. "Record payout" asks for the method, the period, the amount, the date received and the reference.
4. Every money value through the existing formatting helper, never typed by hand.

Follow `docs/DESIGN-SYSTEM.md` .

9. Tests

Accounts

1. Creating an account with an opening balance of 120000 writes one `OPENING` entry, and `outstandingFor` returns 120000.
2. Golden day B09: Masala Tea at 50% off with reason `STAFF_OFFICE` , charged to "E-210 Office", leaves the bill `ON_ACCOUNT` with `chargedToAccountInPaise` 4700, the table freed, one `CHARGE` entry, and the audit line.
3. Golden day B10: Mexican Bowl and Roasted Papad charged to "W-330 Office" for 50400.
4. A partly paid bill: 20000 paid in cash, then charged, puts only the remainder on the account. C3 holds for that bill: payments plus charged equals bill total.
5. Paying or discounting an `ON_ACCOUNT` bill is refused.

6. Voiding B10 after charging writes `CHARGE_REVERSED` 50400, the outstanding balance goes back to its earlier value, and the order is billable again.
7. Golden day section 5, with the clock set to 27 Sep 2026 1:15 PM: W-330 Office pays 50400 in cash. The collection's business date is 2026-09-27, the outstanding balance is 0, and E-210 Office still owes 4700.
8. A collection above the outstanding balance is refused with `ACCOUNT_BALANCE_EXCEEDED` . A collection with a platform method is refused.
9. An adjustment by a manager is refused. By the owner, it moves the balance and writes the audit line.
10. The statement over 26 to 27 September shows each entry and a running balance that ends at the outstanding balance.
11. Check C10 by hand in the test: for every account touched, `OPENING + CHARGE - CHARGE_REVERSED - COLLECTION ± ADJUSTMENT` equals `outstandingFor` .

Payouts

12. With Swiggy at 2000 basis points commission, and the golden day's Swiggy payment of 93000, a payout for 26 Sep with 74400 received shows expected 74400 and difference 0.
13. A payout covering a payment made before a commission was set lists that payment under "rate not set", and leaves it out of expected.
14. A second payout overlapping the first is refused. After voiding the first, it is accepted.
15. Changing the method's commission later does not

change the expected amount of an existing payout.

16. Every role in both permission tables.

Run the full suite at the end. Every test that passed before must still pass.

10. Non-negotiable rules that apply

"Store all money as whole paise integers."

"Never hard delete a bill, order, or stock entry." Account entries and payouts are never edited or deleted. Mistakes are reversed or voided.

"Every database record has a `restaurantId`. Every query filters by it."

"Money arithmetic lives in `server/utls/money.js`."

Commission maths uses `applyBasisPoints`.

"Schema changes are additive."

11. Checks and golden day

C3 now includes `chargedToAccountInPaise`. Test 4 checks it on one bill. P10 builds the full day-level check.

C10 is checked by hand in test 11. P14 moves it into `reconciliationService.js`.

C11 is the payout comparison in test 12.

B09, B10 and the section 5 collection are reproduced to the paisa.

12. Docs to update

1. `docs/PROJECT-STATE.md` :
 1. The date line.
 2. "Current stage": "Next: P10, cash drawer and Day Close."
 3. Module status: M17 note becomes "Delivery orders in P06, payouts in P09."
 4. Every detail where the build differed from the P07 spec, as a decision row, and the spec updated to match.
 5. "What changed recently": a P09 entry at the top, and the oldest moved to the archive.
 2. `docs/prompts/README.md` : mark P09 as Done.
-

13. Out of scope

The day lock on any of these writes. That is P10.
Credit limits on accounts.
Importing a platform's settlement report file.
Any report screen. R6 and R17 come in P15 and P17.

14. Done when

1. `npm test` passes, with before and after counts recorded.
2. `npm run lint` and `npm run build` pass.
3. By hand: charge a bill to "W-330 Office", see the table free up, record a cash collection the next day, and see

the statement balance go to zero.

4. By hand: record a Swiggy payout for a day and see expected and received side by side.

5. Every doc in section 12 is updated.

6. Commits on `main`, one line each, for example:

`add on hold accounts and ledger`

`charge bills to accounts`

`add account collections and statements`

`add platform payouts`

`update docs for p09`

7. Push `main`.

8. Print a short summary: commits, files changed, test counts before and after, every place `ON_ACCOUNT` changed existing behaviour, and every place the build differed from the spec.